

Jimma University

Faculty of Computing and Informatics

Software Engineering Program

Web Design and Programming

Jimma, Ethiopia.

Server-side scripting—PHP

- It is a web technology that **allows custom HTML to be delivered to a client machine** where the code that generates processed on the web server.
- **Server-side language** code remains on the web server. After it has been processed, the **server sends only the output of the script to the user as HTML format**.
- Server-side language brings websites to life in the following ways:
  - Uploading files through a web page
  - Reading and writing to files
  - Displaying and updating information dynamically
  - Sending feedback from your website directly to your mailbox
  - Using a database to display and store information
  - Making websites searchable. Etc…

Introduction to PHP

- PHP stands for **Hypertext Preprocessor**. PHP is currently in its fifth major version, called PHP5.
- PHP is a server-side scripting language (meaning that the scripts are executed on the server) that can be embedded in HTML.
  - Because PHP scripts execute on the server, **PHP can dynamically create the HTML code** that generates the Web page. This allows individual users to see customized Web pages.
  - Web page visitors see the output from scripts**, but not the scripts themselves.**
- PHP is particularly strong in its ability to interact with databases.
  - PHP handles connecting to the database and communicating with it, avoiding the knowledge of technical details for connecting to a database: **i.e. MySQL, PostgreSQL, Oracle, Sybase**.

PHP Syntax

- The PHP code is enclosed in special **start** and **end** processing instructions **<?php and ?>**
  - You can **add PHP code to your web page** by using tags, similar to other tags in the HTML file.
- The PHP code section is enclosed in PHP tags with the following form:
  - **<?php PHP statements ?>**
- For example, you can add the following PHP section to your HTML file.
  - <?php echo "This line brought to you by PHP"; ?>

    <?php echo "<p>Order processed. </p>"; ?>
- Web pages that **contains PHP should be saved with .php extension**

Cont...

- **index.php on the server**

```
<!doctype html>
<html>
<head>
<title>PHP Hello World!</title>
</head>
<body>
<?php
  echo "Hello World!";
  print "<br>";
  echo "Welcome!";
?>
</body>
</html>
```

What shows up in the browser

Hello World!
Welcome!

Client sees this as source of the page

```
<!doctype html>
<html>
<head>
<title>PHP Hello World!</title>
</head>
<body>
Hello World!<br>Welcome!
</body>
</html>
```

Output Statements

- The two most basic ways of displaying output in PHP are **echo and print**. Both can be used either with parentheses or without.
  - **echo outputitem1** **and** **echo (output);**
  - **print output;** **and** **print(output);**
- Example: different ways of **echo and print**

```
echo 123;                    //output: 123
echo "Hello World!";         //output: Hello world!
echo "Hello", "World!";      //output: Hello World!
echo Hello World!; //error, string should be in quotes
print ("Hello world!");       //output: Hello world!
print "Hello world!";         //output: Hello world!
```

PHP supports two types of comments:

- single-line comment by using the # or //
- multi-line comment by using /* this is commment */

The command print is very similar to echo, with two important differences:

Unlike echo, print can accept only one argument.

Unlike echo, print returns a value, which represents whether the print statement succeeded.

Variable Declaration

- In PHP, all variable names should **start with a dollar sign ($).** This tells PHP that it is a variable name.
  - To store information in a variable, use a single equal sign (=).
- For example:

```
$age = 21;
$price = 20.52;
$temperature = -5;
$name = “Clark Kent”;
echo “Your age is $age”;
```

- **PHP is a loosely typed language.** This means that a single variable may contain any type of data.

Cont...

- PHP has a **total of eight commonly used data types:** integers, doubles, Booleans, strings, arrays, objects, NULL, and resources:
  - ***Integers*** are whole numbers, without a decimal point, like 495.
  - ***Doubles*** are floating-point numbers, like *3.14159* or *49.0*.
  - ***Booleans*** have only two possible values: *TRUE* and *FALSE*.
  - ***Strings*** are sequences of characters, like *"PHP is very interesting"*.
  - ***Arrays*** are named and indexed collections of other values.
  - ***Objects*** are instances of programmer-defined classes.
  - ***Resources*** are special variables that hold references to resources external to PHP such as database connections, file handle.
  - ***NULL*** is a special type that only has one value: *NULL*.

Cont...

- **Removing Variables**: You can remove the variable by using this statement: **unset($age);**
  - After this statement, the variable $age no longer exists. If you try to echo it, you get an "**undefined variable**" notice.
- You can **unset more than one variable** at once, as follows:

  unset($age, $name, $address);
- Sometimes you just need to know **whether a variable exists or what type of data is in the variable**.
- Here are some common ways to test variables:
  - **isset($varname)** - true if the variable is set and not null, even if nothing is stored in it.
  - **empty($varname)** - true if the value is 0, is a string with no characters in it, or is not set.
- You can also **test what type of data** is in the variable. **is_integer($var), is_array($var2), is_bool($var) and also gettype($var)**

Constant variable

- Constants are set by using the **define statement**.
- The general format is as follows:

  define("constantname", "constantvalue");

- For example, to set a constant with the weather, use the following statement:

  define("PI", "3.141"); **print(PI); and echo PI;**

  - This statement creates a constant called PI and sets its value to "3.141".
- Unlike variables, **constant names do not begin with a dollar sign ($).**
- **When we echo a constant, we can't enclose it in quotes.**
  - echo "The value of Interest is $", INTEREST;

```php
<?php
// Defining constant
define("SITE_URL", "https://www.tutorialrepublic.com/");

// Using constant
echo 'Thank you for visiting - ' . SITE_URL;
?>
```

Using Operators

- **Assignment Operators (=)**

```
$my_var = 4;
```

- **Arithmetic Operators (+, - , / , *)**

```
$my_var = +4;    $my_var2= -5;
$sum=  $my_var  +  $my_var2;
echo $sum;
```

- **Comparison Operators (==, <=, >=, <, >,!=)**

```
$my_var2 <= $my_var
```

- **Combination Arithmetic & Assignment Operators (.= , /=, %=, +=,-=, etc…)**

```
$my_str .= "hello";          $my_str = $my_str . "hello";
```

- **Pre- and Post-Increment and Decrement operators**

Conditional Statements

- **if Statements:** We can use an if statement to make a decision.
- **if..else statements:** An else statement allows you to define an alternative action to be taken when the condition in an if statement is false.
- **else if Statements:** For many of the decisions we make, there are more than two options.

```php
<?php
$d = date("D");
if($d == "Fri"){
    echo "Have a nice weekend!";
} elseif($d == "Sun"){
    echo "Have a nice Sunday!";
} else{
    echo "Have a nice day!";
}
?>
```

```php
<?php
$name = isset($_GET['name']) ? $_GET['name'] : 'anonymous';
echo $name;
?>
```

Cont...

Switch statements: The switch statement works in a similar way to the if statement, but allows the condition to take more than two values.

```php
<?php
$today = date("D");
switch($today){
    case "Mon":
        echo "Today is Monday. Clean your house.";
        break;
    case "Tue":
        echo "Today is Tuesday. Buy some food.";
        break;
    case "Wed":
        echo "Today is Wednesday. Visit a doctor.";
        break;
    case "Thu":
        echo "Today is Thursday. Repair your car.";
        break;
    case "Fri":
        echo "Today is Friday. Party tonight.";
        break;
    case "Sat":
        echo "Today is Saturday. Its movie time.";
        break;
    case "Sun":
        echo "Today is Sunday. Do some rest.";
        break;
    default:
        echo "No information available for that day.";
        break;
}
?>
```

Using Loops

- Loop statements are designed to enable you to achieve repetitive tasks.
  - A loop will continue to operate until a condition is achieved, or you explicitly choose to exit the loop.
- There are three types of loops:
- *for loop:* Sets up a counter; repeats a block of statements until the counter reaches a specified number
- *while loop:* Sets up a condition; checks the condition, and if it's true, repeats a block of statements until the condition becomes false
- *do..while loop:* Sets up a condition; executes a block of statements; checks the condition; if the condition is true, repeats the block of statements until the condition becomes false

Cont...

- for ($i = 1; $i <= 3; $i++) {

```
    echo "$i. Hello World!<br>";
}
```

- You can use the following nested loop statements:

```
for($i=1; $i<=9; $i++){
  for($j=1; $j<=9; $j++){
      $result = $i * $j;
      echo "$i x $j = $result\n";
  }
}
```

```
$count = 1;
while ($count <= 10) {
    print("count is $count<BR>");
    $count = $count + 1;
}
```

```
$count = 1;
do{
    print("count is $count<BR>");
    $count = $count + 1;
}while ($count <= 10);
```

Cont...

Exercise

1. Write a while loop that displays prime numbers between 1 and 100 using PHP
2. Write a for loop that displays the squared and cubed value of numbers between 0 and 100 using PHP.
3. Write a do while loop that calculates the factorial of a number using PHP.
4. Write a loop that gets the factors of a given number using PHP.

ARRAY

- In addition to numerical indexes, which is use numerical indices to access their values, PHP also supports associative arrays.
- Associative arrays can have almost anything as the array indices(named keys), but typically use strings.

Creating Arrays

- There are different ways to create an array in a PHP script:
  - by assigning a value into one by one (and thereby implicitly creating it),
  - by using the array() construct, or
  - by calling a function that happens to return an array as its value.
- Example
  - $products[0] = “Apple”;
    $products[1] = “Mango”;
  - $fruit = array(“apple”, “orange”, “banana”);

Cont...

- **Associative Arrays:** In an associative array, we can associate any key or index we want with each value.
- An associative array can be viewed as a **list of key/value pairs,** stored as follows:
- Initializing an Associative Array

  $person = array("name" => “John", "occupation" => "engineer", "age" => 30, "special power" => "x-ray vision");

  $fruit=[“first” => “Apple”, “second”=>”Banana”, “third” => “Mango”];
- You can reference specific elements of an associative array using the specific key.
  - Example: to access associative array element

    echo $person['occupation']; //output: engineer

Cont...

Determining the Size of an Array

- You can **find out the size of your array** by using either the count statement or a sizeof statement.

```
$n = count($arrayname);
$n = sizeof($arrayname);
```

array_push()
array_pop()
array_keys()
array_valus()

Built-in Arrays

- PHP has several built-in arrays that you can use when writing PHP scripts. Different types of information are stored in built-in arrays.
  - For example, information about the server (such as headers, paths, and script locations) is stored in an array called $_SERVER

Cont...

- $GLOBALS: it is superglobal associative array that contains all the global variables in the script.
  - Superglobals are built in variable that are always accessible, regardless of the scope without needing to use the global keyword or any special function.
  - For example, if you use the statement, **$testvar = 1**, you can then access the variable as $GLOBALS ['testvar'].
- $ _POST: Contains all data sent to the server using HTTP Post method(if the form uses method="post".)
  - $HTTP_POST_VARS: Same as $ _POST.
- $ _GET: Contains all the variables passed from a previous page as part of the URL. This includes variables passed in a form using method="get".
  - $HTTP_GET_VARS: Same as $ _GET.

Cont...

- $ _COOKIE and $ _SESSION are superglobal arrays used for storing and managing data across multiple page request
- $ _COOKIE: Contains all the cookie variables. To store small pieces of data on the client machine. It is used to remember user preferences or track session over along period
  - $HTTP_COOKIE_VARS: Same as $ _COOKIE.
- $ _SESSION: Contains all the session variables. Unlike cookies, session data is stored on the server and only a session ID is stored in the client browser.
  - $HTTP_SESSION_VARS: Same as $ _SESSION.

Cont...

- $_REQUEST: Contains all the variables together that are **in $_POST, $_GET, and $_SESSION.**
- $_FILES: Contains the names of files that have been uploaded.
  - $HTTP_FILES_VARS: Same as $_FILES.
- $_SERVER: Contains information about your server. Because your Web server provides the information, the information that's available depends on what server you're using.
  - $HTTP_SERVER_VARS: Same as $_SERVER.
- $_ENV: Contains information provided by your operating system, such as the **operating system name, the system drive, and the path to your temp directory.** This info varies depending on your operating system
  - $HTTP_ENV_VARS: Same as $_ENV.

Function in PHP

- A function is a group of PHP statements that perform a specific task.
- You can use the function wherever you need to perform the task.

```
function functionname($argument1, $argument2, ….){
    block of statements;
    return value;
}
```

```
function addNumbers($a, $b){
        $sum = $a +$b;
        return $sum
}
```

Calling a function

- The following line is the simplest possible call to a function:

```
addNumbers(10,4);
```

Variable Scope

❖ **A variable declared within a function remains local to that function.**

❖ PHP has fairly simple rules:

- Variables declared inside a function are in scope from the statement in which they are declared to the closing brace at the end of the function. **This is called function scope, and they are called local variables.**
- Variables declared outside of functions are in scope from the statement in which they are declared to the end of the file, **but not inside functions**. **This is called global scope and are called global variables.**
  - ➢ The **keyword global** can be used to manually specify that a **variable defined or used within a function will have global scope.**

Example: local variable

```
<?php
   $number=50;
   function test(){
       $testvariable = "this is a test variable";
        echo “Num is ”, $number;   //output: Num is
   }
   echo "test variable: $testvariable";        //output: test variable:
   echo “Num is ”, $number;                    //output: Num is 50
?>
```

- **The value of the variable $testvariable is not printed.** This is because no such variable exists outside the test() function.
- Similarly, a variable declared outside a function will not automatically be available within it.

Cont...

- If you want a variable created within a function to be global, **we can use the keyword global** as follows:

```
function test(){
        global $var;
        $var = "contents";
        echo "inside the function, \$var = ".$var."<br>";
}
test();
echo "outside the function, \$var = ".$var."<br>";
```

output:

inside the function, $var = contents
outside the function, $var = contents

Cont...

- In PHP, global variables must be **declared global inside a function if they are going to be used in that function**.

```
<?php
        $a = 1;        $b = 2;
        function Sum() {
                global $a, $b;
                $b = $a + $b;
        }
        Sum();
        echo $b; ?>
```

- The above script will output "3".
- By declaring **$a and $b global** within the function, all references to either variable will refer to the global version.

Cont...

- A second way to access variables from the **global scope is to use the special PHP-defined $GLOBALS array.**
- The previous example can be rewritten as:

```
<?php
$a = 1;  $b = 2;
function Sum(){
    $GLOBALS["b"] = $GLOBALS["a"] + $GLOBALS["b"];
}
Sum();
echo $b;
?>
```

Exercise

- Write a function that accepts a number and returns the squared and cubed value of the number.
- Write a function that accepts a number as parameter and then calculates the factorial of the number and return it.

Cookies and Sessions

Cookies

- A cookie is a small piece of information that is retained on the client machine, either in the browser's application memory or as a small file written to the user's hard disk.
- A cookie is **stored by the user's browser** in compliance with a request from **a server or script.**
- A cookie is **often used to identify a user.** Each time the same computer requests a page with a browser, it will send the cookie too.
- You store cookies by using the **setcookie function**. The general format is as follows:

  setcookie("variable", "value");

- For example, the following statement stores the pair city=Jimma in the cookie file on the user's computer: setcookie("city","Jimma");

Cont...

- You can access the cookie using a built-in array called **$_COOKIE.**
- The next Web page can display the information from the cookie by using the following statement.
  - echo "Your home city is ".$_COOKIE['city']; //Your home city is Jimma
- **Setting expiration dates**
  - If you want the information stored in a cookie to remain in a file on the user's computer after the user leaves your Web site, **set your cookie with an expiration time**, as follows:
    - setcookie("variable", "value", expiretime); The **expiretime value sets** the time when the cookie expires.

setcookie("state", "CA", **time()+3600**); #expires in one hour
setcookie("Name", $Name, time()+**(3*86400))** #expires 3 days

- **Deleting a Cookie**
  - Officially, to delete a cookie, you should **call setcookie()** with the **name argument only**:

setcookie("city");

- This approach does not always work well, however, and should not be relied on. It is safest to set the cookie with a date you are sure has already expired:

setcookie("city", "", time()-60);

Session(Cont...)

- A session is the time that a user spends at your Web site.
  - Users may view many Web pages between the time they enter your site and leave it.
- To make session information available, PHP does the following:
  - PHP assigns a session ID number.
    - The number is a really long number that is unique for the user and that no one could possibly guess. The session ID is stored in a PHP system variable named **PHPSESSID**.
  - PHP stores the variables that you want saved for the session in a file on the server.
  - The file is named with the session ID number.
  - PHP passes the session ID number to every page.

Opening and closing sessions

- You should open a session at the beginning of each Web page. Open the session with the **session_start function**, as follows:
  - **session_start();**
- The function first checks for an existing session ID number.
  - If it finds one**, it sets up the session variables.**
    - If it doesn't find one, it **starts a new session by creating a new session ID number**.
- To avoid an error, the **session_start function must be called before any output is sent.**
  - This means that it must be the **first line of code in your program**.
- You may want to restrict your site to users with a **valid user ID and password**.
  - For **restricted sessions that users log into**, you often want users to log out when they're finished.
- **To close a session**, use the following statement wherever you want to close the session:

session_destroy();

Working with Forms

- We use one of two methods to submit form information. The methods pass the form data differently and have different advantages and disadvantages
- ***GET method:***
  - The form data is passed **by adding it to the URL** that calls the form-processing script.
  - For example, the URL may look like this:
    - processform.php?lname=Abebe&fname=Kebede
  - The advantages of this method are **simplicity and speed**.
  - The disadvantages are that
    - less data can be passed and
    - the information is displayed in the browser, which can be a security problem in some situations.

Cont...

- ***POST method:***
  - The form data is passed as a **package in a separate communication** with the processing script.
  - The advantages of this method are **unlimited information passing** and **security of the data.**
  - The disadvantages are the **additional overhead and slower speed**.
- The **form data is available in the processing script** in the PHP built-in arrays.
  - Information from forms that use the **POST method** is available in the built-in array called **$_POST.**
  - If your form uses the **GET method**, the information is available in the array **$_GET**.
- Both types of form information are also stored in an array called **$_REQUEST**.
- You get information from the array by using the **form field name as the array key.**

Cont...

- The syntax is:

  $_POST[“form element name”]

  $_GET[“form element name”]

- For example, suppose that you echo the following field in your form that uses the POST method:

  <input type=’text’ name=’firstName’>;

- The value entered in this textbox can be accessed by

  $_POST[‘firstName’]

  - This contains the text the user typed into the field.

- If the form uses the GET method, the above textbox can be accessed as

  $_GET[“firstName”]

Uploading Files

- You may want users to upload files to your Web site. For example, you may want users to be able to upload resumes to your job-search Web site or pictures to your photo album Web site.
  - You can display a form that allows a user to upload a file by using an HTML form designed for that purpose.
- The general format of the form is as follows:

```
<form enctype="multipart/form-data" action="file.php" method="post">
    <input type="hidden" name="MAX_FILE_SIZE" value="30000">
    <input type="file" name="userfile">
    <input type="submit" value="Upload File">
</form>
```

Cont...

- Notice the following points regarding the form:
  - **The enctype attribute is used in the form tag.**
    - You must set this attribute to multipart/form-data when uploading a file.
  - **A hidden field is included that sends a value (in bytes) for MAX_FILE_SIZE.**
    - If the user tries to upload a file that is larger than this value, it won't upload.
  - The submission method should be POST.
  - **The input field that uploads the file is of type file**.
- Along with the file, information about the file is sent with the form.
  - This information is stored in the PHP built-in array called **$_FILES**.
    - An array of information is available for each file that was uploaded.

Accessing Information About An Uploaded File(Cont…)

- You can obtain the information from the array by using the name of the field.

  $_FILES[‘fieldname’][‘name’] – contains filename

  $_FILES[‘fieldname’][‘type’] – contains type of file

  $_FILES[‘fieldname’][‘tmp_name’] – contains temporary location of file

  $_FILES[‘fieldname’][‘size’] – contains size of file

- For example, suppose you use the following field to upload a file:

  <input type=”file” name=”userfile”>

Reusing Code

- **PHP provides two very simple**, yet very useful, statements to allow you to reuse any type of code.
  - Using ***include* or *require* statement**, you can load a file into your PHP script.
    - ➢The file can contain anything you would normally type in a script including PHP statements, text, HTML tags, etc
- The two functions are identical in every way, except how they handle errors:
  - **include() generates** a warning, but the script will continue execution
  - **require() generates** a fatal error, and the script will stop
- These two functions are used to **create functions, headers, footers, or elements** that will be reused on multiple pages

Using include(Cont...)

- The statements require() and include() are very similar, but some important differences exist in the way they work.
- **include()** allows you to include the code contained in another library or script file inside a php file, just as if the code had been copied and pasted into the php file.

Here's a simple example. Say you have created a simple function called animalSpeak() that returns the sound made by either a cat, a dog, or a mouse:

```
function animalSpeak( $animal ) {
      $sounds = array ( “cat” = > “meow”, “dog” = > “woof”, “mouse” = >“squeak” );
      return $sounds[$animal];
}
echo animalSpeak( “mouse” ) . “ < br / > ”; // Displays “squeak”
echo animalSpeak( “cat” ) . “ < br / > ”; // Displays “meow”
```

- store it in a file called *animal_functions.php*

Cont...

- You might then use the function from within a *mouse.php* script file in the same folder, as follows:

```
<?php
        include( “animal_functions.php” );
        echo animalSpeak( “mouse” ) . “ < br / > ”; // Displays “squeak”;
?>
```

You can then use the same function inside a *cat.php* script file:

```
<?php
        include( “animal_functions.php” );
        echo animalSpeak( “cat” ) . “ < br / > ”; // Displays “meow”;
?>
```

include() is very useful when building large websites. It enables you to write code once and then reuse it wherever you want.

Using require:

❖ The following code is stored in a file named *reusable.php*:

```
<?php
  echo “Here is a very simple PHP statement. <br>”;
?>
```

❖ The following code is stored in a file called *main.php*:

```
<?php
  echo “This is the main file. <br> “;
  require(“reusable.php”);
  echo ”The script will end now. <br>”;
?>
```

Reading Assignment

- **File Input/Output in PHP**
- **Classes in PHP**

…Next Class…

Database Programming